

EMBEDDED SYSTEMS
REAL TIME BATTERY MONITORING
EVENT-DRIVEN SAFETY PROTECTION KERNEL

BACHELOR OF TECHNOLOGY
in
ELECTRONICS AND COMMUNICATION
ENGINEERING

Submitted by

Sudanapalli Srinivas

Srinivas79798@gmail.com

CONTENTS

1. Abstract
2. Introduction to Embedded Systems
3. Literature Review
4. System Overview
5. Pin Configurations
6. Safety Protection Features
7. Event-Driven Architecture
8. Testing and Debugging
9. Code
10. Outputs
11. Conclusion

1.ABSTRACT

The Event-Driven Safety Protection Kernel is an embedded safety controller developed to monitor and protect a multi-cell battery system in real time. The system continuously supervises battery parameters including weak cell voltage, overvoltage conditions, sensor faults, and rapid voltage fluctuations. Based on detected events, the controller activates protective actions such as relay disconnection, buzzer alerts, and LCD warning messages.

Unlike traditional systems that rely on blocking delays, the controller is implemented using a fully non-blocking event-driven architecture based on the `millis()` timer function. This approach enables simultaneous monitoring, faster fault response, and improved system reliability. Additional features such as recovery logic, anti-relay chatter protection, and stable state management ensure safe operation under changing battery conditions.

The project demonstrates the application of embedded systems, battery protection strategies, and real-time event processing techniques used in modern automotive and industrial battery management systems.

2.INTRODUCTION TO EMBEDDED SYSTEMS

Embedded systems are dedicated computing systems designed to perform specific control and monitoring functions within electronic devices. They combine hardware and software to provide efficient, reliable, and real-time operation.

Modern battery-powered systems require continuous monitoring to ensure safety and reliability. Embedded controllers play a critical role in detecting abnormal battery conditions and executing protective actions before faults can cause system damage. Event-driven embedded systems improve responsiveness by reacting immediately to events rather than waiting for sequential program execution.

This project implements a real-time battery protection controller using event-driven programming principles to improve battery safety and system stability.

3.LITERATURE REVIEW

Battery protection systems are essential in electric vehicles, renewable energy systems, and portable electronic devices. Traditional battery monitoring systems often rely on periodic polling and blocking delays, which can reduce responsiveness during fault conditions.

Recent advancements in embedded systems have introduced event-driven architectures that allow multiple tasks to run concurrently without blocking system execution. These systems improve fault detection speed and provide more reliable protection against abnormal battery conditions.

Automotive-grade battery protection systems commonly include overvoltage detection, undervoltage detection, sensor fault identification, relay control, and alarm systems. These concepts form the basis of the Event-Driven Safety Protection Kernel developed in this project.

4.EVENT-DRIVEN SAFETY PROTECTION KERNEL

The Event-Driven Safety Protection Kernel continuously monitors battery cell voltages and system conditions while executing protective actions based on detected events.

The controller identifies:

- Weak Cell Conditions
- Overvoltage Conditions
- Sensor Fault Conditions
- Rapid Voltage Fluctuations

When a fault occurs, the system performs the appropriate safety response including:

- Relay cutoff
- Buzzer activation
- LCD warning display
- Fault state logging

The event-driven architecture ensures that all monitoring and protection tasks operate simultaneously without interrupting system performance.

5. PIN CONFIGURATIONS

Component	Type	Function
Cell 1 Voltage Input	Analog Input	Measures Cell 1 Voltage
Cell 2 Voltage Input	Analog Input	Measures Cell 2 Voltage
Cell 3 Voltage Input	Analog Input	Measures Cell 3 Voltage
Cell 4 Voltage Input	Analog Input	Measures Cell 4 Voltage
Relay Module	Digital Output	Battery Pack Protection
Buzzer	Digital Output	Fault Alarm
LCD Display	I2C Output	Warning Messages
ESP32	Controller	Event Processing

6. SAFETY PROTECTION FEATURES

Weak Cell Detection

The system continuously monitors all battery cells. If any cell voltage falls below the predefined threshold, a weak cell event is generated and the protection system responds accordingly.

Overvoltage Protection

When a cell voltage exceeds the maximum safe operating limit, an overvoltage event is triggered. The relay disconnects the battery pack to prevent damage.

Sensor Fault Detection

Sensor readings outside valid operating ranges are identified as sensor faults. The controller alerts the user and places the system into a safe operating state.

Rapid Voltage Fluctuation Detection

Sudden voltage changes are analyzed to detect unstable battery conditions. Rapid fluctuations generate warning events and activate protective actions.

Relay Protection

Relay operations are protected using anti-relay chatter logic to prevent repeated switching during unstable conditions.

7. EVENT-DRIVEN ARCHITECTURE

The controller uses a non-blocking architecture based entirely on the millis () timer function

Main Tasks

- Battery Voltage Monitoring
- Fault Detection
- Relay Management
- Buzzer Control
- LCD Updates
- Fault Recovery Monitoring

Advantages

- No delay () function used
- Real-time monitoring
- Faster fault response
- Efficient CPU utilization
- Improved system stability

The event-driven scheduler allows all tasks to operate independently while maintaining reliable real-time performance.

8. TESTING AND DEBUGGING

The following tests were performed to validate system functionality:

- Verified battery voltage acquisition.
- Tested weak cell fault detection.
- Tested overvoltage protection response.
- Verified sensor fault detection.
- Tested rapid voltage fluctuation monitoring.
- Verified relay cutoff operation.
- Tested buzzer alert generation.
- Verified LCD warning message display.
- Tested fault recovery mechanism.
- Confirmed anti-relay chatter operation.

- Monitored system performance using Serial Monitor.
- Verified stable non-blocking operation under multiple fault conditions.

9.CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
#define CELL1 34
#define CELL2 35
#define CELL3 32
#define CELL4 33
#define RED_LED 2
#define GREEN_LED 4
#define YELLOW_LED 5
#define BUZZER 18
#define RELAY 19
unsigned long sampleTimer = 0;
unsigned long relayTripTime = 0
const unsigned long SAMPLE_INTERVAL = 500;
const unsigned long RECOVERY_TIME = 5000
float v1, v2, v3, v4;
float pv1 = 0;
float pv2 = 0;
float pv3 = 0;
float pv4 = 0;
String status = "NORMAL";
enum SystemState
{
    NORMAL,
    WEAK_CELL,
    OVERVOLTAGE,
    SENSOR_FAULT,
    RAPID_FLUCTUATION
};

SystemState currentState = NORMAL;

bool relayTripped = false;

float readVoltage(int pin)
{
    int raw = analogRead(pin);
    return (raw / 4095.0) * 3.3;
}

void resetOutputs()
{

```

```

digitalWrite(REDA_LED, LOW);
digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(BUZZER, LOW);
}

void tripRelay()
{
    if (!relayTripped)
    {
        relayTripped = true;
        relayTripTime = millis();

        digitalWrite(RELAY, HIGH);
    }
}

void recoverRelay()
{
    if (relayTripped)
    {
        if (millis() - relayTripTime >= RECOVERY_TIME)
        {
            relayTripped = false;
            digitalWrite(RELAY, LOW);
        }
    }
}

void updateState(SystemState newState)
{
    currentState = newState;

    resetOutputs();

    switch (currentState)
    {
        case NORMAL:

            status = "NORMAL";
            digitalWrite(GREEN_LED, HIGH);

            lcd.clear();
            lcd.setCursor(0, 0);
            lcd.print("SYSTEM HEALTHY");
            break;

        case WEAK_CELL:

```

```
status = "WEAK CELL";  
digitalWrite(RED_LED, HIGH);  
digitalWrite(BUZZER, HIGH);  
tripRelay();
```

```
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("WEAK CELL");  
lcd.setCursor(0, 1);  
lcd.print("CHECK PACK");  
break;
```

case OVERVOLTAGE:

```
status = "OVER VOLTAGE";  
digitalWrite(RED_LED, HIGH);  
digitalWrite(BUZZER, HIGH);  
tripRelay();
```

```
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("OVER VOLTAGE");  
lcd.setCursor(0, 1);  
lcd.print("CUT OFF");  
break;
```

case SENSOR_FAULT:

```
status = "SENSOR FAULT";  
digitalWrite(YELLOW_LED, HIGH);  
digitalWrite(BUZZER, HIGH);  
tripRelay();
```

```
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("SENSOR FAULT");  
lcd.setCursor(0, 1);  
lcd.print("CHECK ADC");  
break;
```

case RAPID_FLUCTUATION:

```
status = "VOLT FLUCT";  
digitalWrite(YELLOW_LED, HIGH);
```

```
lcd.clear();  
lcd.setCursor(0, 0);
```



```

        lcd.print("VOLT FLUCT");
        lcd.setCursor(0, 1);
        lcd.print("UNSTABLE");
        break;
    }
}

void setup()
{
    Serial.begin(115200);

    pinMode(RED_LED, OUTPUT);
    pinMode(GREEN_LED, OUTPUT);
    pinMode(YELLOW_LED, OUTPUT);
    pinMode(BUZZER, OUTPUT);
    pinMode(RELAY, OUTPUT);

    lcd.init();
    lcd.backlight();

    digitalWrite(RELAY, LOW);

    updateState(NORMAL);

    Serial.println("EVENT DRIVEN SAFETY PROTECTION KERNEL");
}

void loop()
{
    if (millis() - sampleTimer >= SAMPLE_INTERVAL)
    {
        sampleTimer = millis();

        v1 = readVoltage(CELL1);
        v2 = readVoltage(CELL2);
        v3 = readVoltage(CELL3);
        v4 = readVoltage(CELL4);

        bool weakCell =
            (v1 < 1.0) ||
            (v2 < 1.0) ||
            (v3 < 1.0) ||
            (v4 < 1.0);

        bool overVoltage =
            (v1 > 3.0) ||
            (v2 > 3.0) ||
            (v3 > 3.0) ||

```

```

(v4 > 3.0);

bool sensorFault =
  (v1 <= 0.05) ||
  (v2 <= 0.05) ||
  (v3 <= 0.05) ||
  (v4 <= 0.05);

bool fluctuation =
  (abs(v1 - pv1) > 0.50) ||
  (abs(v2 - pv2) > 0.50) ||
  (abs(v3 - pv3) > 0.50) ||
  (abs(v4 - pv4) > 0.50);

if (sensorFault)
{
  updateState(SENSOR_FAULT);
}
else if (overVoltage)
{
  updateState(OVERVOLTAGE);
}
else if (weakCell)
{
  updateState(WEAK_CELL);
}
else if (fluctuation)
{
  updateState(RAPID_FLUCTUATION);
}
else
{
  recoverRelay();
  updateState(NORMAL);
}

pv1 = v1;
pv2 = v2;
pv3 = v3;
pv4 = v4;

Serial.println("==== SAFETY STATUS =====");
Serial.print("CELL1: "); Serial.println(v1, 2);
Serial.print("CELL2: "); Serial.println(v2, 2);
Serial.print("CELL3: "); Serial.println(v3, 2);
Serial.print("CELL4: "); Serial.println(v4, 2);
Serial.print("STATE: "); Serial.println(status);
Serial.print("RELAY: ");

```

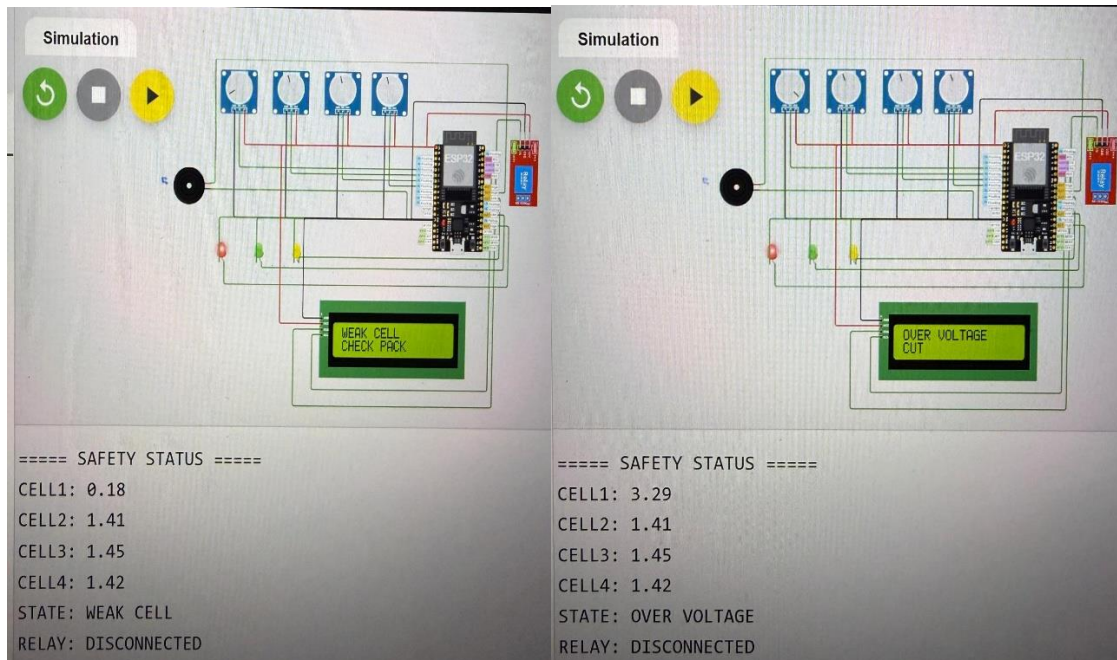
```

if (relayTripped)
    Serial.println("DISCONNECTED");
else
    Serial.println("CONNECTED");

    Serial.println();
}
}

```

10.OUTPUTS



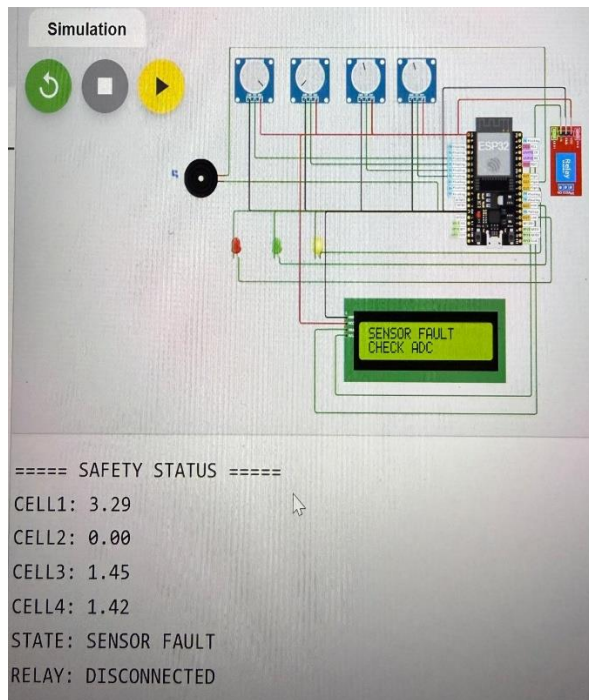


Fig 3 sensor fault

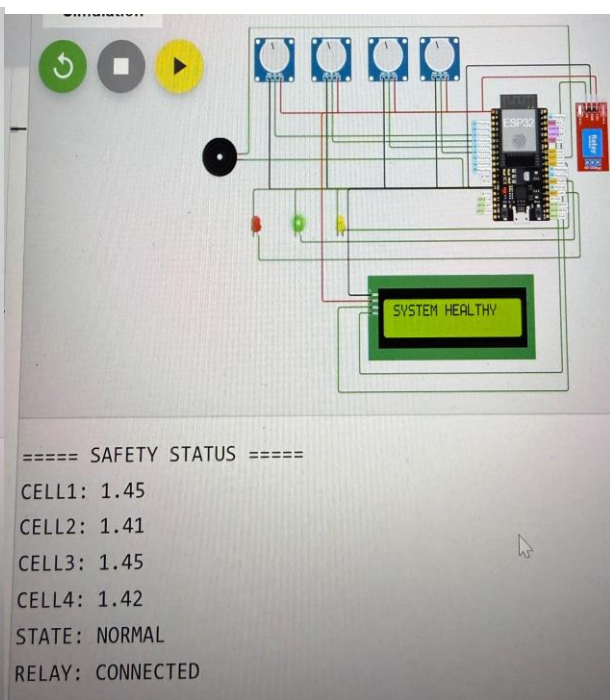


Fig 4 normal/ system healthy

11.CONCLUSION

The Event-Driven Safety Protection Kernel successfully demonstrates a real-time battery protection controller using a fully non-blocking event-driven architecture. The system continuously monitors battery conditions and responds immediately to faults such as weak cells, overvoltage conditions, sensor anomalies, and rapid voltage fluctuations.

The implementation of recovery logic, anti-relay chatter protection, and stable state management improves system reliability and safety. The project highlights the practical application of embedded systems and real-time control techniques used in modern battery management and automotive safety systems.

THANK YOU